Enterprise Microservices Architecture - Complete Summary

📋 Executive Overview

This document provides a comprehensive summary of the enterprise microservices architecture designed for an insurance platform. The architecture follows Domain-Driven Design (DDD), Hexagonal Architecture, and implements comprehensive resilience, security, and observability patterns.

🏗️ Architecture Principles

Core Principles

1. **Domain-Driven Design (DDD)**: Each service owns its bounded context
2. **Hexagonal Architecture**: Ports & Adapters pattern for clean separation
3. **Schema-per-Service**: Database isolation with Oracle schemas
4. **Event-Driven Architecture**: Kafka for async communication
5. **API Gateway Pattern**: Apigee as single entry point
6. **CQRS**: Separate read/write models where needed
7. **Saga Pattern**: Orchestrated via IBM BAW for complex workflows

Technology Stack

- **Language**: Java 21/25
- **Framework**: Spring Boot 3.x/4.x
- **API Gateway**: Apigee (OAuth2/JWT, rate limiting)
- **Message Broker**: IBM Kafka
- **Database**: Oracle (schema-per-service)
- **BPM**: IBM BAW (Business Automation Workflow)
- **Container Orchestration**: AWS EKS
- **Container Registry**: AWS ECR
- **Observability**: AWS OpenSearch, Micrometer
- **Caching**: Redis (via platform-caching-starter)

Resilience Patterns

- Circuit Breakers (Resilience4j)
- Retry with exponential backoff
- Bulkheads for resource isolation
- Timeouts at all integration points
- Idempotency for all operations

Security Principles

- JWT-based authentication (from Apigee)
- Method-level authorization

- Service-to-service authentication
- Correlation ID propagation
- Audit logging for all operations

📦 Platform Framework (8 Starters)

1. platform-parent-bom

Purpose: Dependency management & version alignment

Key Features:

- Centralized version management
- Spring Boot 3.x dependencies
- Resilience4j, Kafka, Jackson versions
- Internal starter version alignment

2. platform-logging-starter

Purpose: Standardized logging across all services

Key Features:

- Correlation ID generation & propagation
- MDC (Mapped Diagnostic Context) utilities
- JSON structured logging (Logback)
- Request/response logging interceptor
- Async thread propagation

3. platform-web-starter

Purpose: Standardize incoming HTTP handling

Key Features:

- Global exception handler
- Bean validation configuration
- Standard error response format
- OpenAPI/Swagger base config
- Controller advice

4. platform-http-client-starter

Purpose: Standardized outbound REST communication + resilience

Key Features:

- Auto-configured WebClient with connection pooling
- Correlation ID propagation to downstream services
- JWT token propagation
- Integrated resilience (retry, circuit breaker, timeout, bulkhead)

- Metrics & distributed tracing
- Property-driven per-service configuration

5. platform-kafka-starter

Purpose: Standardized messaging infrastructure

Key Features:

- Kafka auto-configuration (Producer/Consumer)
- Generic EventEnvelope wrapper
- Dead Letter Topic (DLT) strategy
- Correlation ID propagation
- Observability integration
- Configurable retry & backoff

6. platform-security-starter

Purpose: Standardized authentication & authorization

Key Features:

- JWT validation filter
- Token extraction utilities
- Method security (@PreAuthorize)
- Authentication entry point
- Security exception mapping

7. platform-observability-starter

Purpose: Uniform observability across services

Key Features:

- Micrometer metrics configuration
- Standard metrics naming conventions
- Health indicators
- Distributed tracing (correlation ID)
- Actuator endpoint exposure

8. platform-caching-starter

Purpose: Standardized caching infrastructure

Key Features:

- Spring Cache abstraction
- Redis auto-configuration
- TTL configuration per cache
- Cache metrics integration
- Service-isolated cache keys

🎯 Microservices Architecture (15 Services)

Core Domain Services (5 Services)

1. Policy Management Service

Schema: POLICY_SCHEMA

Purpose: Complete policy lifecycle management

Key Features:

- Policy creation, renewal, endorsement
- Premium calculation integration
- Document management integration
- Workflow orchestration (IBM BAW)

Key Integrations:

- Underwriting Service (risk assessment)
- Billing Service (premium collection)
- Document Service (policy documents)
- Customer Service (customer data)

---

2. Underwriting Service

Schema: UNDERWRITING_SCHEMA

Purpose: Risk assessment and underwriting decisions

Key Features:

- Automated underwriting rules
- Medical requirements management
- Risk scoring & classification
- Underwriter workbench

Key Integrations:

- Rules Service (underwriting rules)
- Document Service (medical reports)
- Policy Service (policy creation)

---

3. Claims Service

Schema: CLAIMS_SCHEMA

Purpose: Claims processing and settlement

Key Features:

- Claims registration & processing
- Document verification
- Settlement calculation
- Fraud detection integration

Key Integrations:

- Policy Service (policy validation)
- Document Service (claim documents)
- Billing Service (settlement payment)
- Rules Service (claims validation)

---

4. Billing & Collections Service

Schema: BILLING_SCHEMA

Purpose: Premium billing and payment collection

Key Features:

- Premium calculation & billing
- Payment processing (multiple modes)
- Grace period management
- Lapse & revival handling

Key Integrations:

- Policy Service (premium details)
- Payment Gateway (external)
- Notification Service (payment reminders)

---

5. Commission Service

Schema: COMMISSION_SCHEMA

Purpose: Agent commission calculation and payment

Key Features:

- Commission calculation (multiple structures)
- Commission statement generation
- Payment processing
- Clawback handling

Key Integrations:

- Policy Service (policy events)
- Billing Service (premium collection)

- Agent Management (external)

Supporting Domain Services (3 Services)

6. Reinsurance Service

Schema: REINSURANCE_SCHEMA

Purpose: Reinsurance treaty and cession management

Key Features:

- Treaty management (quota share, surplus, XOL)
- Automatic cession calculation
- Reinsurer settlement
- Bordereau generation

Key Integrations:

- Policy Service (policy cession)
- Claims Service (claim recovery)
- Reinsurer Systems (external)

---

7. Customer Service

Schema: CUSTOMER_SCHEMA

Purpose: Customer 360° view and management

Key Features:

- Customer profile management
- KYC/AML verification
- Relationship management
- Customer segmentation

Key Integrations:

- Authentication Service (user accounts)
- Document Service (KYC documents)
- External KYC providers

---

8. Product Configuration Service

Schema: PRODUCT_SCHEMA

Purpose: Insurance product catalog and configuration

Key Features:

- Product definition & versioning
- Coverage & rider configuration
- Premium rate tables
- Underwriting rules configuration

Key Integrations:

- Policy Service (product selection)
- Underwriting Service (UW rules)
- Rules Service (product rules)

Infrastructure Services (7 Services)

9. Document Management Service

Schema: DOCUMENT_SCHEMA

Purpose: Document storage, retrieval, and processing

Key Features:

- AWS S3 storage integration
- OCR with AWS Textract
- Document verification workflows
- Version control & retention

Key Integrations:

- AWS S3 (storage)
- AWS Textract (OCR)
- All domain services (document needs)

---

10. Notification Service

Schema: NOTIFICATION_SCHEMA

Purpose: Multi-channel notification delivery

Key Features:

- Email, SMS, Push, WhatsApp support
- Template management
- Bulk notifications
- Delivery tracking & preferences

Key Integrations:

- Email Gateway (SMTP)
- SMS Gateway (Twilio/AWS SNS)
- Push Notification (FCM)

- WhatsApp Business API

---

11. Authentication Service

Schema: AUTH_SCHEMA

Purpose: User authentication and authorization

Key Features:

- OAuth2/JWT token management
- Multi-factor authentication (MFA)
- Role-based access control (RBAC)
- Session management

Key Integrations:

- Apigee (token validation)
- LDAP/AD (enterprise SSO)
- Customer Service (user profiles)

---

12. Audit Service

Schema: AUDIT_SCHEMA

Purpose: Immutable audit trail and compliance

Key Features:

- Event sourcing pattern
- Compliance reporting (GDPR, SOX, HIPAA)
- Audit trail query & export
- Data retention policies

Key Integrations:

- All services (audit events)
- Compliance systems (external)
- Data warehouse (analytics)

---

13. Master Data Service

Schema: MASTER_DATA_SCHEMA

Purpose: Reference data and lookup management

Key Features:

- Code tables (countries, currencies, etc.)

- Hierarchical data (org structure)
- Localization support
- Cache-friendly design

Key Integrations:

- All services (reference data)
- External data providers

---

14. Reporting Service

Schema: REPORTING_SCHEMA

Purpose: Business intelligence and regulatory reporting

Key Features:

- Scheduled report generation
- Ad-hoc query builder
- Dashboard management
- Regulatory compliance reports

Key Integrations:

- All services (data sources)
- AWS OpenSearch (analytics)
- BI tools (Tableau, Power BI)

---

15. Rules Service

Schema: RULES_SCHEMA

Purpose: Business rules engine and decision management

Key Features:

- Rule set management & versioning
- Decision table execution
- Rule testing framework
- BRE adapter integration

Key Integrations:

- Underwriting Service (UW rules)
- Policy Service (eligibility rules)
- Claims Service (validation rules)
- External BRE (complex rules)

🔄 Service Interaction Patterns

Synchronous Communication (REST)

- **Use Case**: Real-time queries, immediate responses needed
- **Pattern**: WebClient with resilience (circuit breaker, retry, timeout)
- **Example**: Policy Service → Customer Service (get customer details)

Asynchronous Communication (Kafka)

- **Use Case**: Event notifications, eventual consistency
- **Pattern**: EventEnvelope with correlation ID
- **Example**: Policy Service publishes PolicyCreated → Billing Service consumes

Orchestration (IBM BAW)

- **Use Case**: Complex multi-step workflows
- **Pattern**: Saga orchestration
- **Example**: Policy issuance workflow (underwriting → approval → issuance)

📊 Event Flow Architecture & Principles

Event Envelope Pattern

All events follow a standardized envelope structure:

```
{
  "eventId": "uuid",
  "eventType": "DomainEventName",
  "timestamp": "2024-02-16T10:00:00Z",
  "source": "service-name",
  "correlationId": "correlation-uuid",
  "payload": {
    // Domain-specific data
  }
}
```

Event Envelope Example

```
{
  "eventId": "uuid",
  "eventType": "PolicyCreated",
  "timestamp": "2024-02-16T10:00:00Z",
  "source": "policy-service",
  "correlationId": "correlation-uuid",
  "payload": { /* domain-specific data */ }
}
```

Event Categories

1. **Domain Events**: Business state changes (PolicyCreated, ClaimApproved)
2. **Integration Events**: Cross-service coordination (DocumentVerified, PaymentReceived)
3. **System Events**: Infrastructure events (ServiceStarted, HealthCheckFailed)

Event Patterns

- **Event Notification**: Inform other services of state changes
- **Event-Carried State Transfer**: Include necessary data in event
- **Event Sourcing**: Audit service stores all events
- **CQRS**: Separate read models updated via events

2. Integration Events

- DocumentVerified, PaymentReceived, etc.
- Cross-service coordination
- Eventual consistency

3. System Events

- ServiceStarted, HealthCheckFailed, etc.
- Infrastructure monitoring
- Operational alerts

Event Design Best Practices

1. Event Naming

- Use past tense (PolicyCreated, not CreatePolicy)
- Be specific (PaymentReceived, not Payment)
- Include entity type (Policy, Claim, etc.)

2. Event Payload

- Include correlation ID for tracing
- Include timestamp for ordering
- Include minimal but sufficient data
- Avoid sensitive data (PII)

3. Event Versioning

- Use semantic versioning (v1, v2)
- Support backward compatibility
- Deprecate old versions gradually

4. Event Ordering

- Use partition keys for ordering
- Same entity events go to same partition
- Handle out-of-order events gracefully

5. Event Idempotency

- Include unique event ID
- Consumers should be idempotent
- Handle duplicate events

🗄️ Database Architecture Principles

Schema-per-Service Strategy

Each microservice has its own Oracle schema with complete data isolation:

- **No cross-schema queries**: Services cannot directly query other schemas
- **Event-driven synchronization**: Use Kafka events for data consistency
- **API-based queries**: Use REST APIs for real-time cross-service data needs
- **Read replicas**: For reporting and analytics (separate concern)

Naming Conventions

- **Schemas**: `{SERVICE}_SCHEMA` (e.g., POLICY_SCHEMA)
- **Tables**: UPPERCASE with underscores (e.g., POLICIES, POLICY_COVERAGES)
- **Columns**: UPPERCASE with underscores (e.g., POLICY_ID, CREATED_AT)
- **Indexes**: `IDX_{TABLE}_{COLUMN}` (e.g., IDX_POLICIES_STATUS)
- **Foreign Keys**: `FK_{TABLE}_{REFTABLE}` (e.g., FK_COVERAGE_POLICY)

Data Types

- **IDs**: VARCHAR2(50) for flexibility
- **Codes**: VARCHAR2(100) for business codes
- **Names**: VARCHAR2(200) for display names
- **Amounts**: NUMBER(15,2) for monetary values
- **Dates**: DATE for date-only, TIMESTAMP for date-time
- **Flags**: CHAR(1) with 'Y'/'N' values
- **JSON**: CLOB for complex structures

Schema-per-Service Strategy

Each service has its own Oracle schema:

- POLICY_SCHEMA
- UNDERWRITING_SCHEMA
- CLAIMS_SCHEMA
- BILLING_SCHEMA
- COMMISSION_SCHEMA
- REINSURANCE_SCHEMA
- CUSTOMER_SCHEMA
- PRODUCT_SCHEMA
- DOCUMENT_SCHEMA
- NOTIFICATION_SCHEMA

- AUTH_SCHEMA
- AUDIT_SCHEMA
- MASTER_DATA_SCHEMA
- REPORTING_SCHEMA
- RULES_SCHEMA

Best Practices

1. Indexing Strategy

- Primary key indexes (automatic)
- Foreign key indexes (for joins)
- Status column indexes (for filtering)
- Date column indexes (for range queries)
- Composite indexes (for common query patterns)

2. Partitioning Strategy

- Time-based partitioning for audit tables
- Range partitioning for large tables
- Monthly intervals for event logs

3. Data Retention

- Hot data: 30 days (online)
- Warm data: 90 days (compressed)
- Cold data: 7 years (archived)

4. Backup Strategy

- Daily full backups
- Hourly incremental backups
- Point-in-time recovery enabled
- Cross-region replication

5. Performance Optimization

- Connection pooling (HikariCP)
- Query optimization (explain plans)
- Index maintenance (rebuild monthly)
- Statistics gathering (weekly)

🔒 Security Architecture

Authentication Flow

1. User authenticates via Apigee (OAuth2)
2. Apigee validates credentials and issues JWT
3. JWT contains user identity, roles, permissions

4. Services validate JWT using platform-security-starter
5. Method-level authorization via @PreAuthorize

Service-to-Service Authentication

1. Internal JWT for service-to-service calls
2. Propagated via platform-http-client-starter
3. Short-lived tokens (15 minutes)
4. Automatic refresh mechanism

Security Layers

- **API Gateway**: Rate limiting, IP whitelisting, OAuth2
- **Service Layer**: JWT validation, method security
- **Data Layer**: Row-level security, encryption at rest
- **Network Layer**: VPC isolation, security groups

📈 Observability Architecture

Metrics

- **Business Metrics**: Policies created, claims processed, etc.
- **Technical Metrics**: Response times, error rates, throughput
- **Infrastructure Metrics**: CPU, memory, disk usage

Logging (AWS OpenSearch)

- **Structured JSON logs**: Consistent format across services
- **Correlation ID**: End-to-end request tracing
- **Log levels**: DEBUG, INFO, WARN, ERROR
- **Retention**: 30 days hot, 90 days warm, 1 year cold

Tracing

- **Correlation ID propagation**: Across all services
- **Request flow visualization**: Service dependency mapping
- **Performance analysis**: Identify bottlenecks

Health Checks

- **Liveness**: Is service running?
- **Readiness**: Can service handle requests?
- **Custom checks**: Database connectivity, Kafka connectivity

---

🎯 Key Design Decisions

1. Why Schema-per-Service?

- **Data isolation**: Each service owns its data

- **Independent scaling**: Services can scale independently
- **Technology flexibility**: Can use different databases if needed
- **Failure isolation**: Database issues don't cascade

2. Why Kafka for Events?

- **Decoupling**: Services don't need to know about each other
- **Scalability**: Handle high throughput
- **Reliability**: Message persistence and replay
- **Event sourcing**: Audit trail and debugging

3. Why Hexagonal Architecture?

- **Testability**: Easy to mock external dependencies
- **Flexibility**: Swap implementations without changing core logic
- **Maintainability**: Clear separation of concerns
- **Domain focus**: Business logic isolated from infrastructure

4. Why Platform Starters?

- **Consistency**: Same patterns across all services
- **Productivity**: Faster development
- **Maintainability**: Centralized updates
- **Best practices**: Enforced standards

🏥 Business Domain Scope

Group Business

Key Processes:

1. Pre-Issuance Servicing
2. Pre-issuance Servicing – NML Changes
3. Informal Group – STP Flow / Bank
4. Informal Group – STP flow – Group Portal
5. Informal Group UW (non-STP flow)
6. Post Issuance Servicing
7. Facultative Reinsurance
8. WegaPlus Integration

Key Sub-Processes (26 items):

- BRE Integration
- Auto Assignment
- Flow-based rules
- User-based Authority (Workflow)
- Authority-based decisions
- XRT Calculator Integration
- Document view

- Authority-based referrals
- Workflow Dedupe/BRD Integration
- STP vs Non-STP case flows
- Tele/Medical/Financial Requirements
- Underwriting case flow
- Role & Authority management
- Allocation rules
- One-page UW view (DRS)
- TAT & Escalation
- UW Summary
- UW Guidelines references
- Digital CDF Generation
- UW Referrals (Reinsurer, Sr. UW, HoD, CMO, Risk)
- Counteroffer cases

Pre-Issuance Changes - Member Related (13 items):

- DOB, Gender, Name changes (LA, Nominee)
- Contact info (Mobile, Email)
- PAN, Pin-Code
- Height/Weight, Annual Income

Pre-Issuance Changes - Contract Related (13 items):

- Loan Amount, Disbursement Date, Share
- Master Code
- Policy Term, Rider (Add/Delete/Modify)
- Sum Assured changes
- LAN changes

Post-Issuance Changes - Corrections (24 items):

- Document verification flow (16 items)
- Underwriting flow (8 items)
- Includes: Email, Mobile, Address, PAN, DOB, Gender, Name, Nominee, Rider, Joint Life, RCD/LDD/PAD/PRD backdating

Post-Issuance Changes - Surrenders & Cancellations:

- Document verification for Surrenders
- Document verification for Cancellations

Retail Business

Key Processes:

1. Pre-Issuance
2. Underwriting
3. Issuance

4. Post Issuance

Key Sub-Processes (25 items):

- Digidrive Integration
- Application Tracker Integration
- COPS & UW Auto Allocation
- Pre-Issuance servicing
- Policy Issuance
- UW Authority Limits
- **UW Digital Rating Sheet** (18 sub-components):
    - Application/Applicant details
    - Case journey
    - ICICI Prudential Risk Analytics integration
    - UDS Document view
    - UW Remarks, Audit Trail
    - Pre/Post Issuance servicing sections
    - XRT Calculator Integration
    - Grievance flow
    - UW Chatbot Integration
    - UW Checklist
    - Pre-Policy, Medicals, Financial details
    - BRE section
    - URE Integration (if applicable)
- Pre-Login process
- STP vs Non-STP flows
- Tele/Medical/Financial Requirements
- **UW Internal Referrals** (8 types):
    - Reinsurer, Sr. UW, HoD, CMO, Risk, Accuity, IT, COPS
- Accuity Tool Integration
- Risk System Integration
- HoD Email Integration
- Reinsurer Portal Integration
- Validations in workflow
- Counteroffer cases
- Medical Management flow
- ECamp & Communication Engine Integration
- TPA Portal Integration
- Requirement flow (Medical & Financial)

Pre-Issuance Service Changes (21 items):

- Product, Address, Pin Code changes
- Customer Declaration
- Frequency changes
- Rider Add/Delete
- ATS level changes (Branch/FSC)
- PT/PPT Increase/Decrease

- Sum Assured Increase/Decrease
- Gender, DOB changes
- Residential status
- Email, Mobile, PAN, Aadhar corrections
- Account details

Post-Issuance Service Changes (20 items):

- Policy alteration, Product change
- Premium/SA/Term Increase/Decrease
- Rider Addition/Change
- Add/Delete life
- Assignment (with/without consideration)
- Proposer/Joint Life/Payor changes
- Mis-selling Product Change
- Death Benefit/PPT/Policy Term Increase
- Nomination/Appointee
- Disclosure post-issuance
- LA/Proposer DOB, Residential status
- Top-DB Increase

🔗 External System Integrations

Third-Party Systems (from diagram):

- NSDL, UIDAI, Credit Score, EPFO, CERSAI, Reinsurers
- IIB (Integration Bus)
- Litmus, AA (Account Aggregator)
- DigiDrive, App Tracker, Ineo
- Life Asia, Group Asia, DigiDrive, Accuity, iONEG, WegaPlus, Elitmass, BRE
- LMS, FileNet, SPAARC, IIB, PTLR, PTLL, Posidex, Email Campaign
- Tele Armsoft, FIRCO, Reinsurer URE, DRC

Internal Systems:

- IBM BAW (Workflow Engine)
- Rule Engine
- Schedulers/Jobs
- Oracle Application DB, Archive DB, ADW
- Reporting
- Log Management (Logging, Archival)
- Dev Ops (Code Repo, Maven, SonarQube, Test)
- Container Orchestration (AWS EKS)
- Container Repository (AWS ECR)

Security & Monitoring:

- Security (Penetration Testing, VAPT/SAST/DAST, Secrets Manager)

- Infra & Security (APM, Process Monitoring, Communication Engine, Ticketing Tool, Project Mgmt Tool)